

Replayable Batch-Comparison Audit Transcripts

Gio Della-Libera

Draft — June 28, 2026

Abstract

Batch-comparison audits are useful when ballots are tabulated and hand-tallied as batches rather than linked ballot by ballot. RCOUNT’s V.18 surface verifies that a sampled batch’s reported subtotals reconcile to the package batch summary, that hand-tallied subtotals are present, that the declared plurality margin overstatement follows from reported and hand totals, and that a derived algorithm run carries the matching overstatement taint. The replay layer then routes `batch-comparison-v1` through the shared Kaplan-Markov taint product transcript in integer parts-per-million arithmetic. The result is a narrow public-evidence contract: whole-batch reported subtotals and hand tallies must support the declared taint-product transcript, while ballot-level linkage and legal audit sufficiency remain outside this slice.

1 Introduction

Batch-comparison audits sit between ballot-level comparison and aggregate manual checks. The sampled unit is a whole reported batch: a precinct, scanner batch, mail batch, or other tabulation unit whose reported contest subtotals can be compared with a hand tally of the same batch. This is useful when ballots are only counted or retrievable in batches and ballot-level CVR linkage is not available.

RCOUNT treats this as a replay contract. The verifier does not accept a free text statement that a batch was checked. It confirms that the reported winner and loser subtotals match the canvassed batch summary, that hand-tallied totals exist for the same selections, and that the declared overstatement equals the difference between the reported and hand margins. A derived algorithm transcript then carries the batch overstatement as a taint, encoded as overstatement divided by reported margin.

This paper covers the implemented V.18 slice. Its claim is deliberately narrow: when a package publishes trustworthy batch subtotals, hand tallies, and sampled batch order, the taint-product transcript for `batch-comparison-v1` can be regenerated from those public records.

2 Batch Contract

The package-level record is `BatchComparisonAudit`. It carries the fields needed to compare one audited batch against the batch summary and hand tally:

Field	Meaning
<code>audit_id</code>	Stable id for the batch comparison record.
<code>contest_id</code>	Contest whose batch subtotals are being checked.
<code>batch_id</code>	Audited batch; it must exist in the package batch manifest.
<code>declared_batch_ballots</code>	Declared batch size, checked against <code>counted_ballots</code> .
<code>winner_selection_id, loser_selection_id</code>	Pairwise plurality assertion.
<code>reported_totals</code>	Reported batch subtotals for the assertion selections.
<code>hand_totals</code>	Hand-tallied batch subtotals for the same selections.
<code>declared_reported_margin</code>	Reported winner-minus-loser margin in the batch.
<code>declared_hand_margin</code>	Hand-tallied winner-minus-loser margin in the batch.
<code>declared_overstatement</code>	Reported margin minus hand margin.

For the positive synthetic fixture, the reported assertion margin is five votes and the hand margin is three votes, so the declared overstatement is two votes. The derived algorithm run stores this as the rational taint $2/5$ in the corresponding `AuditSampleStep.assorter_value`. The replay surface then passes the ordered taints to the shared Kaplan-Markov taint-product routine. That routine validates the risk limit, requires at least one taint, rejects taints greater than or equal to one, multiplies the running product by $1 - \text{taint}$, and emits the running p-value in integer parts per million.

3 Implemented Surface

The method id is `batch-comparison-v1`. In `rcount-audit`, the algorithm dispatcher routes that id to `replay_comparison_taint_product_run`. The same replay function is also used by the Kaplan-Markov taint-product path when the MACRO fields are not present. For batch comparison, a missing `risk_limit_ppm` produces the explicit boundary message “batch-comparison taint-product replay requires risk_limit_ppm” rather than a trusted replay.

The core verifier runs two separate checks. First, `verify_batch_comparison_audits` verifies the batch evidence: unique audit ids, existing batch ids, stable batch size, reported totals against canvassed batch summaries, hand totals for both assertion selections, reported margin, hand margin, and overstatement. The helper `check_reported_batch_total` is the narrow reconciliation check for reported winner and loser subtotals.

Second, `verify_audit_algorithm_runs` adds `batch_comparison_algorithm_linkage` when a `batch-comparison-v1` run appears with matching batch comparison audits. That linkage check recomputes each sampled batch taint as `declared_overstatement / declared_reported_margin` and compares it to the declared sample-step assorter value. The helper `derive_batch_comparison_algorithm` builds that run from verified batch comparison records and a supplied sampled-batch order.

Check	Failure caught
<code>batch_comparison_overstatement</code>	Missing batch, batch-size drift, reported-total drift, missing hand tally, or declared margin and overstatement drift.
<code>batch_comparison_algorithm_linkage</code>	Missing sampled-batch evidence, invalid reported margin, assertion mismatch, empty sample, or taint drift between the batch record and algorithm step.
<code>audit_algorithm_transcript</code>	Unsupported method id, invalid risk limit or p-value fields, and the replayable algorithm-run envelope.
<code>source_hash_match</code>	Source evidence bytes still match the declared source index before audit records rely on them.

4 Fixtures

The package-level positive fixture is `synthetic_batch_comparison_package`. It extends the mail-batch fixture with one `BatchComparisonAudit` for `batch:P-001:election-day`. The record declares 70 batch ballots, reported totals of 35 for `cand-a` and 30 for `cand-b`, hand totals of 34 and 31, a reported margin of five, a hand margin of three, and an overstatement of two. The package passes with a `batch_comparison_overstatement` check.

The algorithm positive fixture is `synthetic_batch_comparison_algorithm_package`. It calls `derive_batch_comparison_algorithm_run` with the sampled batch order containing `batch:P-001:election-day`, risk limit 300000, and a `Continue` decision. The derived sample step carries taint $2/5$ and source references from the batch comparison audit and its hand-tally source. The fixture exercises both the package verifier linkage check and the CLI `replay-audit-algorithms` path.

Fixture	What it catches
<code>synthetic_bad_batch_comparison_package</code>	Edits <code>declared_overstatement</code> ; the recomputed reported-minus-hand margin disagrees.
<code>synthetic_missing_hand_tally_batch_comparison_package</code>	Removes the loser hand tally; the overstatement cannot be computed from public hand evidence.
<code>synthetic_batch_size_drift_comparison_package</code>	Changes <code>declared_batch_ballots</code> ; the check fails before overstatement arithmetic.
<code>synthetic_bad_batch_comparison_algorithm_package</code>	Edits the declared sample-step taint; linkage recomputes the taint from the batch record and fails.

The derivation tests also reject an empty sampled-batch list and a sampled batch with no

matching comparison evidence. These are not statistical disagreements; they are evidence-boundary failures that prevent a replayable transcript from being derived.

5 Boundaries

The implemented claim boundary is the one stated in the V.18 plan: batch comparison is useful when ballot-level linkage is unavailable, but large batches weaken localization and can increase audit workload. RCOUNT can verify that the batch records are internally consistent and that the taint-product transcript matches those records. It does not make a ballot-level comparison claim.

The boundary also depends on input trust. The verifier needs reported batch subtotals that correspond to the canvassed batch summary and hand tallies that come from the same physical batch. If those records are absent, mislabeled, or not trustworthy, the replay surface cannot repair them. The missing-hand-tally, batch-size-drift, missing-sampled-batch, and missing-risk-limit cases therefore fail or produce explicit boundary transcripts rather than statistical comfort.

Finally, this slice does not call the derivation helper from public source adapters. The plan keeps that item open until audit reports provide sampled-batch order. V.18 therefore documents the core replay and verification surface, not a complete jurisdiction-specific ingestion pipeline.

6 Conclusion

RCOUNT now has a concrete batch-comparison audit contract. Package verification checks reported batch totals against canvassed batch summaries, requires hand winner and loser tallies, recomputes reported and hand margins, and verifies the declared overstatement. Algorithm-run verification then links each sampled batch to a taint equal to overstatement divided by reported margin, while `replay_comparison_taint_product_run` replays the resulting `batch-comparison-v1` transcript in integer parts-per-million arithmetic.

The result is intentionally modest. It makes whole-batch comparison evidence replayable when trustworthy batch subtotals, hand tallies, and sampled-batch order are present. It does not infer ballot-level evidence, certify statutory sufficiency, or hide missing public records behind a pass. Those limits are the point: V.18 turns batch-comparison audits into a checkable public transcript without pretending that the transcript is more granular than the evidence.

References

- Gio Della-Libera. Rcount overview: Reproducible election count packages. Technical report, Apportionment Research, 2026a.
- Gio Della-Libera. Batch comparison audits. Technical report, Apportionment Research, 2026b.